MongoDB $regex vs Atlas Search — Performance Benchmark Report

Executive Summary

This report compares query latency between plain MQL `$regex` (collection scan) and MongoDB Atlas Search (`$search / regex`) across a 100,000-document corpus, measured under both single-client sequential load and concurrent Locust load testing. The results show that **the right Atlas Search strategy delivers up to 147× lower latency** for selective pattern matching, while MQL retains a decisive advantage for dense patterns retrieved with a small result limit.

Test Environment

Parameter	Value
Dataset	100,000 synthetic documents
Rare entity records	~600 per type (CRIT, AUDIT, DEPLOY)
Common record types	Log entries, config, API responses, access logs, stack traces
Atlas Search index	`lucene.keyword` on `content`; `lucene.standard` multi-field on `content.std`
Sequential benchmark	60 measured iterations · 15 warm-up · 5 queries/iter = 300 samples/op
Concurrent load tool	Locust 2.43 — ~9–10 req/s, `constant_pacing(1)` per virtual user
Metrics reported	Avg, Median, Min, Max (sequential) · P50/P95/P99 (Locust)
Failures	**0% in both runs**

MQL `$regex` Results

All queries use a plain collection scan (COLLSCAN). No text index is involved.

Latency (ms)

Query	Pattern type	Limit	P50	P95	P99
`date_prefix`	Dense, anchored	20	**1**	11	270
`ip_subnet`	Dense, anchored	20	**2**	8	93
`log_level_warn`	Dense, unanchored	20	**2**	7	130
`multi_level`	Dense, alternation	20	**2**	10	22
`mql_not_regex`	Negation	20	**1**	5	10
`crit_code`	**Selective**, complex	all	**1,000**	1,700	1,800
`audit_op`	**Selective**, complex	all	**1,000**	1,600	1,700

Key Observations

- **Dense patterns with `LIMIT 20` are fast** (P50 = 1–2 ms) because MongoDB stops scanning after finding 20 matches. This is not an index benefit — it is early termination on a warm buffer pool.
- **Selective patterns are uniformly slow** at ~1,000 ms P50 regardless of percentile. Only ~600 matching documents exist in 100K, so the engine must scan the entire collection to confirm no further matches. This cost is irreducible without an index.
- **P99 tail for dense patterns** reaches 22–270 ms, revealing occasional contention under concurrent load even when the median looks healthy.

Atlas Search `$search` Results

Three strategies tested: plain regex, compound (text pre-filter + regex), and text-only.

Latency (ms)

Query	Strategy	Pattern / Token	P50	P95	P99
`search_regex/date_prefix`	Plain regex	`2024-0[1-9]-.*`	**2**	16	57

Query	Strategy	Pattern / Token	P50	P95	P99
`search_regex/ip_subnet`	Plain regex	`192\.168\..*`	**2**	17	230
`search_regex/log_level_warn`	Plain regex	`.*WARN.*`	**91**	260	330
`compound/crit_compound`	Compound	token=CRIT, regex=`CRIT-10[0-9]{2}.*`	**16**	56	120
`compound/audit_compound`	Compound	token=AUDIT, regex=`.*AUDIT-EVENT.*`	**460**	840	1,400
`compound/deploy_compound`	Compound	token=DEPLOY, regex=`.*DEPLOY.*`	**420**	860	1,100
`text/text_crit`	Text only	CRIT	**12**	45	78
`text/text_audit`	Text only	AUDIT	**12**	69	98
`text/text_deploy`	Text only	DEPLOY	**490**	1,000	1,300
`search_mustNot/not_error`	mustNot regex	`.*ERROR.*`	**6**	32	540

Key Observations

- **Anchored prefix regex** (`2024-0[1-9]-`, `192\.168\.`) allows Lucene to navigate the term dictionary directly — P50 = 2 ms, comparable to MQL with a limit.
- **Leading `.*` regex** (`.*WARN.*`, `.*AUDIT-EVENT.*`) forces a full term dictionary scan even inside a compound query. `log_level_warn` rises to P50 = 91 ms vs 2 ms for MQL with limit.
- **`compound/crit_compound` is the benchmark winner**: text pre-filter reduces candidates from 100K to ~600, and the anchored regex processes those 600 in microseconds — P50 = **16 ms**.
- **`text_deploy` is slow** (P50 = 490 ms) because "deploy" also appears as a username in access log entries, reducing the token's selectivity from ~600 to ~10,000 documents. Token quality directly controls pre-filter effectiveness.

Sequential Single-Client Benchmark

Setup: 100,000 docs · 60 measured iterations · 300 samples/op · default fetch: top 20 Single client, no concurrency — isolates query engine cost from network/scheduling effects.

Dense Patterns — top 20 results

Scenario	Method	Avg	Median	Min	Max
Simple keyword	`$regex` MQL	**1.4 ms**	1.3 ms	1.2 ms	3.3 ms
	`$search` regex	60.5 ms	58.3 ms	52.8 ms	82.1 ms
	`$search` compound	61.8 ms	59.4 ms	53.4 ms	85.8 ms
	`$search` **text ★**	**4.4 ms**	3.9 ms	2.7 ms	10.2 ms
Date pattern	`$regex` MQL	**1.2 ms**	1.2 ms	1.1 ms	1.9 ms
	`$search` regex	61.1 ms	58.7 ms	54.5 ms	85.7 ms
Case-insensitive	`$regex` MQL	**1.5 ms**	1.4 ms	1.3 ms	3.2 ms
	`$search` regex	59.1 ms	57.1 ms	52.1 ms	91.9 ms
	`$search` compound	58.1 ms	56.4 ms	52.5 ms	80.2 ms
Alternation	`$regex` MQL	**1.2 ms**	1.2 ms	1.1 ms	1.6 ms
	`$search` regex	62.5 ms	60.4 ms	55.5 ms	85.5 ms
	`$search` compound	64.3 ms	62.8 ms	56.3 ms	90.6 ms
IP address	`$regex` MQL	**1.7 ms**	1.3 ms	1.2 ms	4.9 ms
	`$search` regex	73.6 ms	62.3 ms	52.6 ms	137.8 ms
JSON field (port)	`$regex` MQL	**1.4 ms**	1.2 ms	1.2 ms	3.3 ms
	`$search` regex	57.9 ms	51.9 ms	50.9 ms	101.2 ms

Dense pattern takeaway: MQL wins by 40–53× for plain regex with top-20 limit. The `$search` **text operator** is the exception — 4.4 ms median, only 3× behind MQL — because it uses an

inverted-index term lookup, not a regex scan. Atlas Search regex carries a fixed ~52–55 ms floor (mongot IPC overhead) that MQL does not pay.

Selective Patterns — all hits (~600 matching docs)

Scenario	Method	Avg	Median	Min	Max	Hits
CRIT on prod-db-01	`$regex` MQL	768.1 ms	766.4 ms	631.5 ms	964.4 ms	113
	`$search` regex	102.8 ms	101.7 ms	100.1 ms	122.1 ms	113
	`$search` **compound ✓**	**101.8 ms**	100.9 ms	99.8 ms	111.5 ms	113
Audit DELETE events	`$regex` MQL	809.3 ms	795.5 ms	652.6 ms	984.4 ms	136
	`$search` regex	6.2 ms	5.9 ms	5.4 ms	8.7 ms	136
	`$search` **compound ✓**	**5.5 ms**	5.4 ms	4.9 ms	7.9 ms	136
Deploy svc-api/prod	`$regex` MQL	772.1 ms	776.7 ms	614.8 ms	958.0 ms	53
	`$search` regex	55.8 ms	54.1 ms	52.8 ms	78.9 ms	53
	`$search` **compound ✓**	**55.2 ms**	53.3 ms	52.3 ms	94.9 ms	53

⚡ **Headline result: Audit DELETE compound — 5.5 ms vs 809 ms MQL = 147× faster.** This is the maximum gain achievable when the text pre-filter token is highly specific (very few matching docs) and the regex pattern is processed over a tiny candidate set.

Negation Patterns — top 20 results

Scenario	Method	Avg	Median	Min	Max
Negate ERROR	`$not` `$regex` MQL	**1.0 ms**	1.0 ms	0.8 ms	1.8 ms
	`$search` mustNot	4.9 ms	4.6 ms	4.4 ms	11.8 ms

MQL is 4.9× faster for negation with a small limit. Both are operationally acceptable; the Atlas Search mustNot adds fixed IPC overhead with no index benefit for this pattern.

Head-to-Head Comparison

Numbers combine both test methods; single-client sequential benchmark used for selective patterns (most stable signal); Locust P50 used for concurrent load comparison.

Scenario	MQL	Atlas Search best	Improvement	Winning strategy
Selective — high-specificity token + pattern	795 ms	**5.5 ms**	**147×**	compound + anchored regex
Selective — moderate specificity	767 ms	**101 ms**	**7.5×**	compound + anchored regex
Selective — anchored regex, concurrent load	1,000 ms (P50)	**16 ms** (P50)	**63×**	compound + anchored regex
Dense — top 20, anchored pattern	**1–2 ms**	1–2 ms	Tie	MQL `$regex` + limit
Dense — full-text keyword, top 20	**1.4 ms**	4.4 ms (`$search text`)	Atlas 3× slower	MQL wins; text is viable
Dense — Atlas regex, top 20	**1.2 ms**	58–73 ms	Atlas 40–53× slower	MQL `$regex` + limit
Dense — unanchored, concurrent	**2 ms** (P50)	91 ms (P50)	Atlas 45× slower	MQL `$regex` + limit
Negation — top 20	**1.0 ms**	4.9 ms	MQL 4.9× faster	MQL `$not $regex` + limit

---

Recommendations

1. Use Atlas Search compound for selective entity lookups

The ideal pattern — a rare token combined with an anchored regex — delivers up to **147× lower latency** than MQL. This is the primary migration target.

```javascript
// Best case: selective token + anchored prefix regex
{ $search: { index: "content_search", compound: {
    filter: [{ text: { path: { value: "content", multi: "std" }, query: "AUDIT
DELETE" } }],
    must:   [{ regex: { path: "content", query: "AUDIT-EVENT.*DELETE.*" } }]
}}}
```

The two levers that multiply the gain:

- **Token specificity** — the rarer the token, the smaller the candidate set the regex runs on. `AUDIT DELETE` (two tokens) narrows more aggressively than `AUDIT` alone.
- **Regex anchoring** — a pattern starting with a fixed literal prefix (`CRIT-10`, `AUDIT-`) lets Lucene skip the bulk of the term dictionary. A leading `.*` forces a full scan.

2. Consider `$search text` as a drop-in for dense keyword queries

The sequential benchmark shows `$search text` at **4.4 ms** for a dense keyword — only 3× behind MQL's 1.4 ms, but with the scalability and ranking features of Atlas Search. It outperforms Atlas Search regex (60 ms) by **14×** for the same query and does not carry the ~52 ms mongot IPC floor that regex imposes.

```javascript
// Atlas Search text — fast for dense keyword patterns, no regex overhead
{ $search: { index: "content_search",
    text: { path: { value: "content", multi: "std" }, query: "WARN" }
}}
```

3. Keep MQL `$regex` for dense patterns with a small limit

Atlas Search regex has an inescapable ~52 ms minimum latency (mongot IPC round-trip). MQL `$regex` with `LIMIT 20` on a warm buffer pool delivers 1–2 ms by stopping early. There is no index-based alternative that beats MQL here for top-N retrieval. Do not migrate these queries.

4. Avoid leading `.*` in Atlas Search regex patterns

Every pattern in this benchmark starting with `.*` incurred a full Lucene term dictionary scan, negating index benefit. When substring matching is unavoidable:

- Combine with a text pre-filter to reduce the document candidate set.
- Prefer a pure `text` query if the token alone is precise enough.
- If `.*` is unavoidable and the token is not selective, MQL may remain faster.

5. Validate token selectivity before building compound queries

A text pre-filter is only effective when its token is genuinely rare. In this benchmark "deploy" appeared as a username in unrelated access log entries, expanding the candidate set from ~600 to ~10,000 and degrading performance proportionally. Before relying on a compound query in production, verify token frequency with:

```javascript
db.collection.aggregate([
  { $searchMeta: { index: "content_search",
      text: { path: { value: "content", multi: "std" }, query: "<token>" }
  }}
])
```

If the count is unexpectedly high, refine the token or filter on an additional field.

Atlas Search Fixed Overhead — Key Context

The sequential benchmark reveals that every Atlas Search query carries a **minimum ~52 ms latency floor** regardless of index efficiency. This is the mongot inter-process round-trip cost (Atlas Search runs in a separate process from mongod). Implication:

- For any query completing in < 52 ms with MQL, Atlas Search regex **cannot win on raw latency** even with a perfect index hit.
- Atlas Search earns its keep on **selective full-corpus scans** where MQL must read all 100K documents (400–800 ms) — the 52 ms floor is irrelevant at that scale.
- The text operator partially avoids this floor (4.4 ms observed), likely due to a lighter result pipeline when `returnStoredSource` is combined with a small result set.

Summary Decision Matrix

Scenario	**Recommended approach**
Selective entity lookup — rare token + anchored regex	**Atlas Search compound** (**up to 147×**)
Selective entity lookup — rare token, no clean prefix	Atlas Search compound (2–14× gain; accept `.*` cost)
Dense pattern — top-N results, anchored	**MQL `$regex` + limit** (40–53× faster than Atlas regex)
Dense keyword — top-N, no regex required	MQL `$regex` + limit; `$search text` viable (3× gap)

Scenario	Recommended approach
Full-text ranked search	**Atlas Search text operator**
Negation — top-N only	**MQL** `$not` `$regex` **+ limit** (5× faster)
Token with poor selectivity	Fix schema / token design before choosing engine